```
define i8 @f(i8 %x) {
entry:
  ret i8 %x
}
```

$\sqsubseteq$

```
define i8 @f(i8 %x) {
entry:
  %x = add i8 %x, 0
  ret i8 %x
}
```

We do not model the **SSA** IR

$x \sqsubseteq x + 0$

i.e., x refines $x + 0$

Verified Peephole Optimizer for Arithmetic Expressions in Lean 4

Hongxu Xu

hongxu.xu@uwaterloo.ca

Peephole Optimization

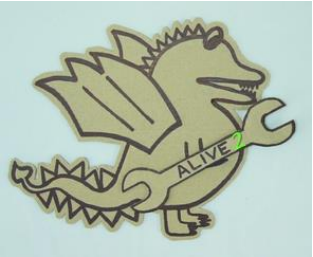
- Optimize code in a **small window** (peephole)

$$x + 0 \rightarrow x$$

```
define i8 @f(i8 %x) {
entry:
  ret i8 %x
}
```

$\sqsubseteq$

```
define i8 @f(i8 %x) {
entry:
  %x = add i8 %x, 0
  ret i8 %x
}
```



Alive2

provides a **translation validation** tool to perform **refinement,** check between two LLVM IR functions

based on **Z3** (an SMT Solver)

COMPILER EXPLORER

Add... More

LLVM IR source #1

LLVM IR

```llvm
define i32 @src(i32 %x) {
  ret i32 %x
}

define i32 @tgt(i32 %x) {
  %res = add i32 %x, 1
  ret i32 %res
}
```

alive-tv (Editor #1, Compiler #2) LLVM IR

alive-tv

Compiler options...

```
----------------------------------------
define i32 @src(i32 %x) {
#0:
  ret i32 %x
}
=>
define i32 @tgt(i32 %x) {
#0:
  %res = add i32 %x, 1
  ret i32 %res
}
Transformation doesn't verify!

ERROR: Value mismatch

NOTE: The counterexample is unique.

Example:
i32 %x = #x00000003 (3)

Source:

Target:
i32 %res = #x00000004 (4)
Source value: #x00000003 (3)
Target value: #x00000004 (4)
```

https://alive2.llvm.org/ce/z/5RpnJi

Undef

represents a value that can be any value when being evaluated

```
define i32 @src1() {
    ret i32 undef
}
```

```
define i32 @src2() {
    ret i32 undef
}
```

```
define i32 @src3() {
    ret i32 undef
}
```

```
define i32 @tgt1() {
    ret i32 0
}
```

```
define i32 @tgt2() {
    ret i32 1
}
```

```
define i32 @tgt3() {
    ret i32 -100
}
```

https://alive2.llvm.org/ce/z/jLhp4e

Parameter Values Can Be Undef

```
define i8  @src(i8  %x) {
   %res = mul i8  %x , 2
   ret i8  %res
}
```

```
define i8  @tgt(i8  %x) {
   %res = add i8  %x , %x
   ret i8  %res
}
```

https://alive2.llvm.org/ce/z/_RdUm6

Parameter Values Can Be Undef

different **undef**s can be evaluated to different values, even if they come from the same source (parameter)

```
define i8  @src(       ) {
   %res = mul i8  undef, 2
   ret i8  %res
}
```

≠

```
define i8  @tgt(       ) {
   %res = add i8  undef, undef
   ret i8  %res
}
```

https://alive2.llvm.org/ce/z/S_zl9l

Parameter Values Can Be Undef

different **undef**s can be evaluated to different values, even if they come from the same source (parameter)

```
define i8  @src(       ) {
   %res = mul i8  undef, 2
   ret i8  %res
}
```

$\not\equiv$

```
define i8  @tgt(       ) {
   %res = add i8  undef, undef
   ret i8  %res
}
```

`%res` can be any **even** i8 numbers

`%res` can be any i8 numbers

Odd numbers are unexpected!

https://alive2.llvm.org/ce/z/S_zl9l

Alive2 Cannot Prove This!

Alive2 **timed out** while processing your query.

```
define i32 @src(        ) {
   %res = mul i32 undef, 2
   ret i32 %res
}
```

```
define i32 @tgt(        ) {
   %res = add i32 undef, undef
   ret i32 %res
}
```

https://alive2.llvm.org/ce/z/kLDrSq

But We Can!

even more general

Not

```
theorem undef_mul2_is_not_refined_by_undef_add_generic (w : Nat) (hpos : 0 < w) :
  ¬ ([llvm-1-definition|
      define i$0 @f() {
        entry:
          %x = add i$0 undef, undef
          ret i$0 %x
      }
    ].instantiateWidths (singletonWidths w)
    ⊑
    [llvm-1-definition|
      define i$0 @f() {
        entry:
          %x = mul i$0 undef, 2
          ret i$0 %x
      }
    ].instantiateWidths (singletonWidths w)) := by
```

But We Can!

even more general

```
theorem undef_mul2_is_not_refined_by_undef_add_generic (w : Nat) (hpos : 0 < w) :
  ¬ ([llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = add i$0 undef, undef
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = mul i$0 undef, 2
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)) := by
```

Integer with an **arbitrary width** represented by a **metavariable**

But We Can!

even more general

```
theorem undef_mul2_is_not_refined_by_undef_add_generic (w : Nat) (hpos : 0 < w) :
  ¬ ([llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = add i$0 undef, undef
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = mul i$0 undef, 2
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)) := by
```

Instantiate the **width metavariable** with **w**

But We Can!

even more general

```
theorem undef_mul2_is_not_refined_by_undef_add_generic (w : Nat) (hpos : 0 < w) :
  ¬ ([llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = add i$0 undef, undef
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑   Refines
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = mul i$0 undef, 2
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)) := by
```

Refinement

... forall args ...

```
def Definition.Refines (y x : @AST.Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
```

Refinement

... such that everything is well-formed ...

```
def Definition.Refines (y x : @AST.Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
        WellFormed x →
        WellFormed y →
        SignatureCompatible x y →
        ArgValuesWellFormed x args →
```

Refinement

... two conditions are satisfied

```
def Definition.Refines (y x : @AST.Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
        WellFormed x →
        WellFormed y →
        SignatureCompatible x y →
        ArgValuesWellFormed x args →
        ((
```

Condition 1: **Definedness**

```
        ) ∧ (
```

Condition 2: **Value Refinement**

```
        ))
```

Refinement

Condition 1: Definedness
For all possible undef assignments, if x can be evaluated to a value, y should too

```
def Definition.Refines (y x : @AST.Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
        WellFormed x →
        WellFormed y →
        SignatureCompatible x y →
        ArgValuesWellFormed x args →
        ((
            (∀ (u : UndefChain), ∃ (ret_x : RegisterValue),
                run (eval x args) default ⟨u, 0⟩ = .ok ret_x) →
            (∀ (u' : UndefChain), ∃ (ret_y : RegisterValue),
                run (eval y args) default ⟨u', 0⟩ = .ok ret_y)
            ) ∧ (

                Condition 2: Value Refinement

        ))
```

Refinement

Condition 2: Value Refinement
For all values returned by y, it should refine the value returned by x

```
def Definition.Refines (y x : @AST.Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
        WellFormed x →
        WellFormed y →
        SignatureCompatible x y →
        ArgValuesWellFormed x args →
        ((
            (∀ (u : UndefChain), ∃ (ret_x : RegisterValue),
                run (eval x args) default ⟨u, 0⟩ = .ok ret_x) →
            (∀ (u' : UndefChain), ∃ (ret_y : RegisterValue),
                run (eval y args) default ⟨u', 0⟩ = .ok ret_y)
            ) ∧ (
            ∀ (u : UndefChain) (ret : RegisterValue),
                run (eval y args) default ⟨u, 0⟩ = .ok ret →
                ReturnValuesRefined x args u ret
        ))
```

Return Value Refinement

```
def Definition.ReturnValuesRefined (x : @Definition ϕ) (args : List RegisterValue)
    (_u : UndefChain) (ret : RegisterValue) : Prop :=
    ∃ u',
        run (eval x args) default ⟨u', 0⟩ = .ok ret
∨
∃ ret_x,
        run (eval x args) default ⟨u', 0⟩ = .ok ret_x ∧
        RegisterValue.Refines ret ret_x
```

Equivalent

Or

Refined

```
def RegisterValue.Refines : RegisterValue → RegisterValue → Prop
| .bv w_x _, .bv w_y .poison => w_y = w_x
| .bv w_x (.value v_x), .bv w_y (.value v_y) => ∃ h : w_y = w_x, h ▸ v_y = v_x
| .void, .void => True
| _, _ => False
```

How to Prove

```
theorem undef_mul2_is_not_refined_by_undef_add_generic (w : Nat) (hpos : 0 < w) :
  ¬ ([llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = add i$0 undef, undef
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = mul i$0 undef, 2
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)) := by
```

```
intro h
```

```
have haddRun :
    run (eval ([llvm-1-definit
        define i$0 @f() {
        entry:
            %x = add i$0 undef
            ret i$0 %x
        }
    ].instantiateWidths (singl
        ({ supplyChain := [1,
    Except.ok (RegisterValue.b
        simp [simp_llvm]
        rw [<- BitVec.add_zero 1#w]
        simp
```

1 goal

```
w : ℕ
hpos : 0 < w
h : AST.Definition.instantiateWidths (singletonWidths w)
    ([llvm-definition|define i$0 @f() {
    entry:
        %x = add i$0 undef! 0, undef! 1
        ret i$0 %x
    }]) ⊑
  AST.Definition.instantiateWidths (singletonWidths w)
    ([llvm-definition|define i$0 @f() {
    entry:
        %x = mul i$0 undef! 0, 2
        ret i$0 %x
    }])
⊢ False
```

```
have haddRun :
    run (eval ([llvm-1-definition|
        define i$0 @f() {
        entry:
            %x = add i$0 undef, undef
            ret i$0 %x
        }
    ].instantiateWidths (singletonWidths w)) []) default
        ({ supplyChain := [1, 0], supplyIndex := 0 }) =
    Except.ok (RegisterValue.bv w (.value (BitVec.ofNat w 1))) := by
        simp [simp_llvm]
        rw [<- BitVec.add_zero 1#w]
        simp
```

Evaluating RHS (**y**) with **1st undef = 1, 2nd undef = 0**

```
have haddRun :
    run (eval ([llvm-1-definition|
        define i$0 @f() {
        entry:
            %x = add i$0 undef, undef
            ret i$0 %x
        }
    ].instantiateWidths (singletonWidths w)) []) default
        ({ supplyChain := [1, 0], supplyIndex := 0 }) =
    Except.ok (RegisterValue.bv w (.value (BitVec.ofNat w 1))) := by
        simp [simp_llvm]
        rw [<- BitVec.add_zero 1#w]
        simp
```

Result is 1

```
have haddRun :
    run (eval ([llvm-1-definition|
        define i$0 @f() {
        entry:
            %x = add i$0 undef, undef
            ret i$0 %x
        }
    ].instantiateWidths (singletonWidths w)) []) default
        ({ supplyChain := [1, 0], supplyIndex := 0 }) =
    Except.ok (RegisterValue.bv w (.value (BitVec.ofNat w 1))) := by
        simp [simp_llvm]
        rw [<- BitVec.add_zero 1#w]
        simp
```

All **evalXXX** definitions are annotated with **[simp_llvm]**

```
    rw [<- BitVec.add_zero 1#w]
    simp
```

The refinement

```
obtain ⟨u', hu'⟩ := (h []
    (by simp [simp_wellform])
    (by simp [simp_wellform])
    (by rfl)
    (by simp [Definition.ArgValu
).2 [1, 0] (RegisterValue.bv w
```

```
h : AST.Definition.instantiateWidths (singletonWidths w)
    ([llvm-definition|define i$0 @f() {
    entry:
        %x = add i$0 undef! 0, undef! 1
        ret i$0 %x
    }]) ⊑
  AST.Definition.instantiateWidths (singletonWidths w)
    ([llvm-definition|define i$0 @f() {
    entry:
        %x = mul i$0 undef! 0, 2
        ret i$0 %x
    }])
```

```
cases u' with
| nil =>
    have hnil :
        run (eval ([llvm-1-defir
            define i$0 @f() {
```

Specialize h with empty args

and proofs for well-formedness

```
obtain ⟨u', hu'⟩ := (h []
    (by simp [simp_wellform])
    (by simp [simp_wellform])
    (by rfl)
    (by simp [Definition.ArgValuesWellFormed, simp_wellform])
).2 [1, 0] (RegisterValue.bv
```

```
cases u' with
| nil =>
    have hnil :
        run (eval ([llvm-1-def
            define i$0 @f() {
            entry:
                %x = mul i$0 u
                ret i$0 %x
```

```
def Definition.Refines (y x : @Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
    WellFormed x →
    WellFormed y →
    SignatureCompatible x y →
    ArgValuesWellFormed x args →
    ((
        (∀ (u : UndefChain), ∃ (ret_x : RegisterValue),
        run (eval x args) default ⟨u, 0⟩ = .ok ret_x) →
        (∀ (u' : UndefChain), ∃ (ret_y : RegisterValue),
        run (eval y args) default ⟨u', 0⟩ = .ok ret_y)
        ) ∧ (
        ∀ (u : UndefChain) (ret : RegisterValue),
        run (eval y args) default ⟨u, 0⟩ = .ok ret →
        ReturnValuesRefined x args u ret
    ))
```

```
obtain ⟨u', hu'⟩ := (h []
    (by simp [simp_wellform])
    (by simp [simp_wellform])
    (by rfl)
    (by simp [Definition.ArgValuesWellFormed, simp_wellform])
).2 [1, 0] (RegisterValue.bv w (.value (BitVec.ofNat w 1))) haddRun
```

Obtain the 2nd condition

```
cases u' with
| nil =>
    have hnil :
        run (eval ([llvm-1-
            define i$0 @f()
            entry:
                %x = mul i$0
                ret i$0 %x
            }
```

```
def Definition.Refines (y x : @Definition ϕ) : Prop :=
    ∀ (args : List RegisterValue),
    WellFormed x →
    WellFormed y →
    SignatureCompatible x y →
    ArgValuesWellFormed x args →
    ((
        (∀ (u : UndefChain), ∃ (ret_x : RegisterValue),
        run (eval x args) default ⟨u, 0⟩ = .ok ret_x) →
        (∀ (u' : UndefChain), ∃ (ret_y : RegisterValue),
        run (eval y args) default ⟨u', 0⟩ = .ok ret_y)
        ) ∧ (
        ∀ (u : UndefChain) (ret : RegisterValue),
        run (eval y args) default ⟨u, 0⟩ = .ok ret →
        ReturnValuesRefined x args u ret
    ))
```

```
obtain ⟨u', hu'⟩ := (h []
    (by simp [simp_wellform])
    (by simp [simp_wellform])
    (by rfl)
    (by simp [Definition.ArgValuesWellFormed, simp_wellform])
).2 [1, 0] (RegisterValue.bv w (.value (BitVec.ofNat w 1))) haddRun
```

Specialize it with

- undef supply chain = `[1, 0]`
- `ret = 1`
- A prop stating that RHS (y) returns `ret`

```
| nil =>
    have hnil :
        run (eval ([llvm-1-
            define i$0 @f()
            entry:
                %x = mul i$0
                ret i$0 %x
            }
```

```
def Definition.Refines (y x : @Definition ϕ) : Prop :=
  gs : List RegisterValue),
  ormed x →
  ormed y →
  tureCompatible x y →
  luesWellFormed x args →
  ((
    (∀ (u : UndefChain), ∃ (ret_x : RegisterValue),
    run (eval x args) default ⟨u, 0⟩ = .ok ret_x) →
    (∀ (u' : UndefChain), ∃ (ret_y : RegisterValue),
    run (eval y args) default ⟨u', 0⟩ = .ok ret_y)
    ) ∧ (
    ∀ (u : UndefChain) (ret : RegisterValue),
    run (eval y args) default ⟨u, 0⟩ = .ok ret →
    ReturnValuesRefined x args u ret
  ))
```

Now, we got

- u’, the supply chain for LHS (x)
- hu’, a prop stating that by evaluating LHS (x) under the undef supply u’, either
  - LHS (x) also returns `ret`, or
  - the return value (`ret_x`) of LHS (x) is refined by `ret`

```
obtain ⟨u', hu'⟩ := (h []
    (by simp [simp_wellform])
    (by simp [simp_wellform])
    (by rfl)
    (by simp [Def
).2 [1, 0] (Regis
```

```
def Definition.ReturnValuesRefined (x : @Definition ϕ) (args : List RegisterValue)
    (_u : UndefChain) (ret : RegisterValue) : Prop :=
    ∃ u',
      run (eval x args) default ⟨u', 0⟩ = .ok ret
    ∨
    ∃ ret_x,
      run (eval x args) default ⟨u', 0⟩ = .ok ret_x ∧
      RegisterValue.Refines ret ret_x
```

```
cases u' with
| nil =>
    have hnil :
        run (eval ([llvm-1-
            define i$0 @f()
            entry:
                %x = mul i$0
```

```
        ∀ (u : UndefChain) (ret : RegisterValue),
        run (eval y args) default ⟨u, 0⟩ = .ok ret →
        ReturnValuesRefined x args u ret
    ))
```

Case analysis on the supply chain

```
cases u' with
| nil =>
    have hnil :
        run (eval ([llvm-1-definition|
            define i$0 @f() {
            entry:
                %x = mul i$0 undef, 2
                ret i$0 %x
            }
        ].instantiateWidths (singletonWidths w)) []) default
            ({ supplyChain := [], supplyIndex := 0 }) =
        Except.error "" := by
            simp
            rfl
    simp_all
```

```
cases u' with
| nil =>
    have hnil :
        run (eval ([llvm-1-definition|
            define i$0 @f() {
            entry:
                %x = mul i$0 undef, 2
                ret i$0 %x
            }
        ].instantiateWidths (singletonWidths w)) []) default
            ({ supplyChain := [], supplyIndex := 0 }) =
        Except.error "" := by
            simp
            rfl
    simp_all
```

When the supply chain is empty, LHS will throw an error, because there is an undef value in LHS expecting a supply.

So, the hypothesis is false
and can be easily completed by `simp_all`

```
| cons x xs =>
    have hmulRun :
        run (eval ([llvm-1-definition|
            define i$0 @f() {
            entry:
                %x = mul i$0 undef, 2
                ret i$0 %x
            }
        ].instantiateWidths (singletonWidths w)) []) default
            ({ supplyChain := x :: xs, supplyIndex := 0 }) =
        Except.ok (RegisterValue.bv w (.value ((BitVec.ofNat w x) * (2#w)))) := by
            simp [simp_llvm]
            rfl
simp_all
have hcontra := bitvec_mul_two_ne_one_generic hpos (BitVec.ofNat w x)
contradiction
```

When the supply chain contains at least one element x,

LHS returns $x * 2$

After `simp_all`, hu' is simplified to $x * 2 = 1$

which contradicts a lemma saying that $x * 2 \neq 1$

Summary

https://github.com/xuhongxu96/lean-nano-llvm

Achievements

- an AST for a LLVM-like IR
- a DSL embedded in Lean
  - with unexpanders
- well-formedness checks
- big-step operational semantics
- a supply-chain model of undef
- a semantic refinement relation
- some example theorem
  - including width-generic examples

Limitations for Future Work

- No memory model
  - load and store
  - pointer reasoning
- No optimizer function
  - or relevant proofs
- No control flow
  - only support a single basic block
- Incomplete undef support
- Incomplete instruction support